

Dokumentasi Logic Perhitungan Dashboard SiMarsel

Panduan lengkap cara hitung setiap angka, formula yang dipakai, sumber data, dan istilah penting di dashboard analytic marketing & sales SiMarsel. Ditulis dengan bahasa non-teknis supaya bisa dimengerti tim non-IT.

Untuk: Tim PT. Cerebrum Eduknesia Nusantara

Versi: 1.0 — Mei 2026

Daftar Isi

1. Konsep Dasar & Sumber Data
2. Pemrosesan Data Mentah
3. Ringkasan Performa (Overview)
4. Analisa Bulanan
5. Strategi & Target
6. KPI per Orang
7. Tasklist
8. Insight Hasil (Hasil Sosmed)
9. Konten Hub (Scrip)
10. Analisa Sosmed
11. Produk (Packages)
12. Kode Promo
13. Optimasi Harga
14. Kalender Marsel
15. Glosarium

1. Konsep Dasar & Sumber Data

SiMarsel adalah dashboard analytic untuk monitoring performa marketing & sales. Semua angka yang muncul di dashboard berasal dari database Supabase (PostgreSQL). Tidak ada angka yang dikira-kira atau dihardcode — semuanya hasil hitungan dari data nyata.

Tabel-tabel utama di database

Nama Tabel	Isi	Kolom Penting
transactions	Catatan setiap transaksi pembayaran	trx_id, source_app, revenue, payment_date, email, methode_name, p
downloaders	Jumlah download per app per hari	date, source_app, count
target_config	Target bulanan per app	app_name, year_month, target_sales, target_downloader, target_pren
apps_snapshot	Snapshot konfigurasi app per user	user_id, apps (JSON)
tasks	Daftar tugas tim	title, status, due_date, due_time, assigned_to
monthly_performance	Snapshot performa bulanan (dari upload Excel)	upload_file, source_app, total_sales, conversion_rate, ...
promo_code_rules	Aturan klasifikasi kode promo per user	user_id, code, category, app_name
content_scripts	Skrip konten yang dibuat	title, platform, status, info_skrip, editor
insight_hasil	Hasil analisa insight sosmed	app_name, platform, tanggal, tayangan, jangkauan, interaksi_konten
kpi_divisions, kpi_cards, kpi_portals	KPI per divisi & per individu	name, target, achievement, bobot, perspektif

Konsep penting yang dipake di seluruh aplikasi

1. Kolom tanggal untuk filter periode

Setiap transaksi punya 3 kolom tanggal yang mungkin diisi: **payment_date** (kapan duit benar-benar masuk), **transaction_date** (kapan order dibuat), dan **created_at** (kapan baris dibuat di DB). SiMarsel pakai prioritas:

```
payment_date > transaction_date > created_at
```

Kenapa payment_date duluan? Karena revenue baru dianggap masuk pas duitnya beneran cair. Kalau order dibuat 31 Mei tapi transfer baru masuk 1 Juni → tetap dihitung sebagai pendapatan Juni, bukan Mei.

2. Normalisasi nama app

```
normalizeApp("jadian") = "JADIASN"  
normalizeApp(" jadi BUMN ") = "JADI BUMN"  
(uppercase + buang spasi awal/akhir)
```

Tujuannya supaya "jadian", "JadiAsn", "JADIASN" dianggap sama. Kalau ada inconsistency ngetik di Excel/n8n, tetap match.

3. Normalisasi email (untuk user premium)

```
normalizeEmail("Budi@Gmail.com") = "budi@gmail.com"  
normalizeEmail(" budi@gmail.com ") = "budi@gmail.com"  
(lowercase + buang spasi awal/akhir)
```

■ **Catatan:** Email "Budi@Gmail.com" dan "budi@gmail.com" dianggap orang yang sama. Tujuannya supaya 1 customer cuma dihitung 1 user walaupun penulisan email beda-beda.

2. Pemrosesan Data Mentah

Sebelum dipakai untuk menghitung apa-apa, setiap baris transaksi diolah dulu di sisi browser. Proses ini menambah field baru yang sudah "siap pakai" untuk filter dan grouping.

Untuk setiap baris transactions

Field yang ditambahkan	Cara hitung
parsed_payment_date	Date object dari kolom payment_date (atau fallback)
year	Tahun dari parsed_payment_date (mis. 2026)
month	Bulan 1-12 dari parsed_payment_date
quarter	Kuartal 1-4 dari parsed_payment_date
year_month	Format "YYYY-MM" (mis. "2026-05")
hour	Jam 0-23 dari parsed_payment_date (waktu lokal)
revenue	Number(revenue ?? Revenue ?? total_price) 0

Untuk setiap baris downloaders

Excel original berbentuk "wide" (1 row per tanggal, banyak kolom per app). Saat diolah, di-melt jadi "long" — 1 row per (tanggal × app):

```
Tanggal | JADIASN | JADIBUMN // format wide (Excel)
2026-05-01 | 234 | 187
↓
{ date: "2026-05-01", source_app: "JADIASN", count: 234 } // long
{ date: "2026-05-01", source_app: "JADIBUMN", count: 187 }
```

Klasifikasi status (untuk Bulanan)

Status pembayaran diklasifikasi dari kolom status_message / status / payment_status:

Status	Cocokin dengan kata kunci
berhasil	success, berhasil, paid, sukses, complete
pending	pending, menunggu, wait
expired	expired, kadaluarsa, expire
dibatalkan	cancel, dibatalkan, canceled, gagal, failed

Dedupe transactions

Pas upload Excel, baris yang dobel di-skip otomatis. Cara identifikasi dobel:

```
Kalau trx_id tidak kosong → pakai trx_id sebagai pengenalan unik
Kalau trx_id kosong → pakai kombinasi {email, tanggal, nominal}
```

3. Ringkasan Performa (Overview)

Tab pertama yang muncul saat buka SiMarsel. Berisi headline cards + tren chart + breakdown payment method & jam pembelian.

Headline Cards

Total Revenue (Total Pendapatan)

Total Revenue = SUM(revenue) dari semua transaksi yang lolos filter

Filter aktif: app, tahun, bulan, metode pembayaran. Kalau filter kosong dan data mentah belum di-load, ngambil dari view overview_stats di Supabase (cepat ~50ms tanpa download 364K row).

Total Transaksi

Total Transaksi = jumlah baris di tabel transactions (yang lolos filter)

■ **Catatan:** Setiap baris = 1 line item. Kalau 1 order ada 3 item, dihitung 3 transaksi. Ini BEDA dengan "jumlah pesanan" (distinct trx_id).

Total Downloader

Total Downloader = SUM(count) dari semua row downloaders yang lolos filter

Unique Buyers

Unique Buyers = COUNT(DISTINCT email)
Email di-trim dulu, baris dengan email kosong di-skip

AOV (Average Order Value)

AOV = Total Revenue ÷ Total Transaksi
(= 0 kalau Total Transaksi 0)

Artinya: rata-rata nilai 1 line item.

User Repeat Order

Definisi: email yang punya ≥ 2 pesanan berbeda (distinct trx_id) dalam scope filter saat ini.

```
SELECT count(*) FROM (  
  SELECT email FROM transactions  
  WHERE email != "" AND trx_id != ""  
  [+ filter aktif]  
  GROUP BY email  
  HAVING count(DISTINCT trx_id) >= 2  
)
```

Contoh: Budi punya 3 trx_id berbeda di Januari. Andi cuma 1 trx_id. Yang dihitung user repeat order: **1** (Budi saja).

Konversi

Konversi (%) = (Total Transaksi ÷ Total Downloader) × 100
(= 0 kalau Downloader 0)

Warna: <5% Low (merah), 5-20% Fair (cyan), ≥20% Good (hijau).

Chart Trend (Harian / Mingguan / Bulanan)

Untuk setiap tanggal:

revenue = SUM(revenue) transaksi di tanggal itu
transactions = COUNT transaksi di tanggal itu

```
downloader = SUM(count) downloaders di tanggal itu  
conversion = (transactions ÷ downloader) × 100
```

Breakdown Metode Pembayaran

```
Grup by methode_name:  
value = SUM(revenue) atau COUNT(rows)  
(tergantung toggle "by Revenue" atau "by Jumlah")  
Baris dengan methode_name kosong dilabel "Unknown"
```

Waktu Pembelian (per Jam)

```
Bucket berdasarkan jam (waktu lokal):  
00-05 → "Dini Hari (00-06)"  
06-11 → "Pagi (06-12)"  
12-17 → "Siang/Sore (12-18)"  
18-23 → "Malam (18-00)"  
value = jumlah transaksi di bucket
```

4. Analisa Bulanan

Tab khusus untuk upload Excel bulanan + breakdown per app + per status. Data disimpan ke tabel monthly_performance di database.

Metric per app (setelah dedupe)

Metric	Cara hitung
total_transaksi	COUNT baris setelah dedupe, di-group per source_app (lowercase+trim)
total_berhasil	COUNT baris dengan status berhasil di app itu
total_pending	COUNT baris dengan status pending
total_expired	COUNT baris dengan status expired
total_dibatalkan	COUNT baris dengan status dibatalkan
conversion_rate	$(\text{total_berhasil} \div \text{total_transaksi}) \times 100$ — disimpan dengan 2 angka di belakang koma
total_sales	SUM(nominal) HANYA dari baris dengan status berhasil
harga rata rata	$\text{total_sales} \div \text{total_berhasil}$ (= 0 kalau total berhasil 0)

■■ **Penting:** Yang masuk ke **total_sales** cuma yang statusnya "berhasil". Transaksi yang masih pending/expired/dibatalkan TIDAK ikut dihitung pendapatan. Conversion_rate tetap menghitung semua status.

Mode filter status

User bisa pilih mode tampil: **all** (semua), **berhasil**, **pending**, atau **cancel**. Filter ini pre-process baris sebelum agregasi. Hanya mode "berhasil" yang menampilkan revenue.

Warna conversion rate

Conversion Rate	Warna
$\geq 70\%$	■ Emerald (sangat baik)
50% - 70%	■ Amber (sedang)
< 50%	■ Rose (perlu perbaikan)

5. Strategi & Target

Tab paling kompleks. Tempat user mengatur target bulanan per app, tracking pencapaian harian, dan ngeliat rekapitulasi semua platform.

Bagian-bagian di tab ini

- **Form Target Bulanan** — set target downloader, sales, user premium, konversi per app per bulan
- **Summary per App** — kartu ringkasan pencapaian app yang dipilih
- **Rekapitulasi All Platform** — tabel semua app sekaligus
- **Operational Sheet** — breakdown harian (tabel besar di bawah)

Cara hitung "Real" (actual) harian

Untuk setiap kombinasi **app × tanggal**, sistem menghitung:

Metric Real	Cara hitung
Downloader	SUM(count) dari tabel downloaders untuk app+tanggal itu
Sales	SUM(revenue) dari transactions untuk app+tanggal itu
Premium (User Premium)	COUNT(DISTINCT email) — jumlah USER unik yang transaksi

■■ **Penting:** Premium = **distinct email**, bukan distinct trx_id. Kalau 1 user (1 email) beli 5x → premium = 1, BUKAN 5. Yang dihitung adalah jumlah ORANG, bukan jumlah transaksi.

Manual override

Kalau user isi nilai "Real" manual di cell tabel (mis. karena ada koreksi), nilai itu menang dari hitungan otomatis DB:

```
actualSales = dailyData[tanggal].actualSales ?? SUM(revenue) dari DB
actualDownloader = dailyData[tanggal].actualDownloader ?? SUM(count) dari DB
actualRepeatOrder = dailyData[tanggal].actualRepeatOrder ?? COUNT(DISTINCT email)
```

Daily Target — Flexible / Chained Logic

Target harian gak fix — dia menyesuaikan defisit dari hari-hari sebelumnya. Logikanya: kalau kemarin kurang, hari ini target naik supaya tetap on-track total bulan.

Step 1: target harian dasar

```
baseDailySales = target_sales_bulanan ÷ jumlah_hari_bulan
baseDailyDownloader = target_downloader ÷ jumlah_hari_bulan
baseDailyRepeatOrder = target_premium ÷ jumlah_hari_bulan
```

Step 2: cari hari terakhir yang sudah ada data

Sistem ngecek hari paling belakangan yang punya actual (sales/downloader/premium > 0). Sebut ini lastFilledIdx.

Step 3: hitung defisit untuk hari berikutnya

Untuk hari TEPAT SETELAH lastFilledIdx (= hari yang lagi diisi sekarang), target hariannya naik buat ngejar:

```
totalActualSebelumnya = SUM(actual) dari hari 1 sampai lastFilledIdx
targetSeharusnyaSampaiIni = baseDailySales × (lastFilledIdx + 1)
deficit = targetSeharusnyaSampaiIni - totalActualSebelumnya
flexibleTarget = clamp(baseDaily × 0.2, baseDaily × 2, baseDaily + deficit)
```


Artinya: target fleksibel = base + defisit, tapi DIBATASI minimum 20% dan maksimum 200% dari base.
Biar gak ekstrem.

■ **Tip: Contoh:** target sales bulan ini 30 juta, jadi base harian 1 juta. Hari 1 cuma terealisasi 500rb (kurang 500rb). Hari 2, targetnya jadi: 1 juta + 500rb defisit = 1.5 juta. Kalau hari 1-10 selalu kurang, defisit gak akan bikin target hari 11 jadi 5 juta — dibatasi maksimal 2 juta (2× base).

Step 4: manual override target harian

```
displayTargetSales = dailyData[tanggal].manualTargetSales OR flexibleTargetSales  
(kalau user isi manual, nilai itu yang menang)
```

Status Harian (kolom paling kanan tabel)

Untuk setiap hari, sistem ngasih label status:

Kondisi	Status	Warna
actualSales > 0 dan achievement ≥ 100%	Melebihi	■ Emerald
actualSales > 0 dan achievement 90-99%	Mendekati	■ Amber
actualSales > 0 dan achievement < 90%	Kurang	■ Rose
actualSales = 0 dan tanggal sudah LEWAT (sebelum hari ini)	Kurang	■ Rose
actualSales = 0 dan tanggal hari ini atau masa depan	Menunggu	■ Slate

```
achievement = (actualSales ÷ displayTargetSales) × 100
```

■ **Catatan:** Perubahan terbaru: kalau hari yang LEWAT tidak ada transaksi, sistem hitung real-nya sebagai 0 (defisit penuh). Sebelumnya hari kosong di-skip dari kalkulasi, jadi total selisih sales kelihatan lebih bagus dari realitanya.

Kolom Keterangan (selisih sales harian)

```
kalau actualSales > 0:  
selisih = actualSales - displayTargetSales  
display: "+Rp X" atau "-Rp X"  
  
kalau actualSales = 0 dan tanggal sudah lewat:  
selisih = 0 - displayTargetSales = -displayTargetSales  
display: "-Rp X" (defisit)  
  
kalau actualSales = 0 dan tanggal hari ini/masa depan:  
display: "-" (belum waktunya dihitung)
```

Summary per App (kartu di tengah tab)

Metric	Formula
totalRealDownloader	SUM(actualDownloader) semua hari di bulan ini
totalRealSales	SUM(actualSales) semua hari
totalRealRepeatOrder	SUM(actualRepeatOrder) semua hari
progressDownloader	(totalRealDownloader ÷ target_downloader) × 100
progressSales	(totalRealSales ÷ target_sales) × 100
progressConversion	(totalRealRepeatOrder ÷ totalRealDownloader) × 100

Metric	Formula
selisihSales	totalRealSales – target_sales (signed)
sumKeterangan	SUM kolom "Keterangan" — termasuk defisit dari hari lewat tanpa trx

Rekapitulasi All Platform (tabel semua app)

Sama dengan summary per-app, tapi di-loop untuk semua app yang punya target. Total global juga dihitung:

```

salesProgress (global) = totalRealSales ÷ totalTargetSales × 100
downloaderProgress = totalRealDownloader ÷ totalTargetDownloader × 100
conversionProgress = totalRealRepeatOrder ÷ totalRealDownloader × 100
Konversi target progress = conversionProgress ÷ rataTargetKonversi × 100

```

6. KPI per Orang

Tab KPI berisi kartu performa per individu (sales/marketer/dll), dengan metric per perspektif (mis. Sales, Operasional, Quality). Setiap metric punya target, real, dan bobot.

Hierarki struktur

```
Divisi (mis. "Tim Marketing")
↓
Card per Orang (mis. "Budi - Marketing Officer")
↓
Perspektif (mis. "Sales", "Customer", "Internal Process")
↓
Metric (mis. "Jumlah leads baru", "Conversion rate", ...)
– punya: target, achievement, bobot
```

Formula per metric

Metric	Formula
Pencapaian (%)	$(\text{achievement} \div \text{target}) \times 100$ (= 0 kalau target null/0 atau achievement null)
Score	$(\text{pencapaian} \div 100) \times \text{bobot}$ (bobot dalam %, ideal total per perspektif = 100%)

Agregasi level-atas

Level	Total Score	Total Bobot
Card (per orang)	SUM(score) semua metric di kartu itu	SUM(bobot) semua metric
Perspektif	SUM(score) metric dalam perspektif	SUM(bobot) — ideal = 100%
Divisi (rekap staff)	SUM(score) semua staff	SUM(bobot) semua staff
avgPencapaian Divisi	SUM(pencapaian per metric) ÷ jumlah metric	

■■ **Penting:** Kalau total bobot per perspektif tidak = 100% (toleransi 0.01%), sistem kasih warning. Bobot harus diatur supaya ideal sum 100% per perspektif.

Filter periode KPI

- **filter_year:** All / spesifik tahun / "none" (untuk data legacy yang tidak ada tahun)
- **filter_month:** All / spesifik bulan

Filter "Mine" (hanya KPI sendiri)

```
Untuk role "staf": card.name.toLowerCase().trim() ===
user.email.toLowerCase().trim()
Artinya: nama di card harus persis sama dengan email user (case-insensitive).
```

Parsing angka (penting untuk Excel import)

```
parseLocaleNumber("1.234,56") = 1234.56 // ada koma → titik = ribuan, koma =
desimal
parseLocaleNumber("1234.56") = 123456 // tidak ada koma → titik = ribuan semua
parseLocaleNumber("0.5") = 5 // 0.5 di-treat sebagai "0 lalu 500"
```

■ **Catatan:** Hati-hati: kalau angkanya pecahan kecil (mis. 0.5), TIDAK boleh ada titik desimal tanpa koma — akan dianggap pemisah ribuan. Pakai koma atau hilangkan titiknya.

7. Tasklist

Tab tasklist untuk mengelola tugas tim (Kanban board). Tidak ada perhitungan revenue — hanya menghitung jumlah tugas per status + label urgency deadline.

Status tugas (4 kolom Kanban)

Status	Arti
request	Tugas baru di-request, belum di-assign
todo	Sudah di-assign, belum dikerjakan
progress	Sedang dikerjakan
done	Selesai

Filter "viewMode=mine"

```
Tampil hanya kalau:  
task.assigned_to.trim().toLowerCase() === currentUserEmail.trim().toLowerCase()
```

Label urgency deadline

```
deadline = due_time ada → "due_date due_time"  
due_time gak ada → "due_date 23:59:59"  
  
diffMinutes = (deadline - sekarang) dalam menit  
diffHours = dalam jam  
diffDays = dalam hari kalender
```

Kondisi	Urgency	Warna	Label
status = done	done	■ Emerald	(selesai)
diffMinutes < 0 (sudah lewat)	overdue	■ Rose	"Lewat Xm/Xj/Xh"
diffHours < 24	urgent	■ Rose	"Xj/Xm lagi"
diffDays ≤ 3	soon	■ Amber	"X hari lagi"
Selainnya	normal	■ Slate	tanggal biasa

8. Insight Hasil (Hasil Sosmed)

Tempat input manual hasil performa sosmed per app × platform × tanggal. Disimpan di tabel insight_hasil.

Filter aktif

- app_name (nama app)
- platform (Instagram/TikTok/Facebook/dst)
- dateFrom .. dateTo (rentang tanggal)

Total (kartu summary)

Untuk setiap kolom metric (tayangan, jangkauan, interaksi_konten, klik_tautan, kunjungan, pengikut):

`total = SUM(kolom itu) dari rows yang lolos filter`

ER% (Engagement Rate, chart per tanggal)

Per tanggal:

$ER\% = (SUM(interaksi_konten) \div SUM(jangkauan)) \times 100$

(= 0 kalau jangkauan = 0)

9. Konten Hub (Scrip)

Tab untuk mengelola skrip konten — CRUD tanpa perhitungan revenue. Cuma menghitung jumlah skrip per status.

Status skrip (5 kategori)

Status	Arti
draft	Baru ditulis, belum dicek
review	Sedang di-review/edit
approved	Sudah disetujui, siap dieksekusi
published	Sudah diposting ke platform
all	Filter semua status

Counter per status

```
statusCounts[status] = COUNT skrip di kategori itu (lolos filter)
```

Filter search

Filter teks substring (case-insensitive) di kolom: title, info_scrip, editor, catatan.

10. Analisa Sosmed

Tab analisa konten sosmed yang sudah diposting. Data dari 2 sumber:

- Manual entries di `dailyData[]`.socialContent (lewat Target sheet)
- Skrip konten yang status-nya "published" dari tabel `content_scripts` (metric = 0)

Filter

- `searchTerm` (substring di caption)
- `platformFilter` (Instagram/TikTok/Facebook/dst)
- `jenisFilter` (Reels/Story/Feed/dst)
- `appFilter`
- `dateFrom .. dateTo` (filter tanggalUpload)

Hero totals (kartu summary)

SUM untuk setiap metric:
posts, tayangan, jangkauan, pemirsa,
jumlahBersihInteraksi, sukaTanggapan, komen, share, save,
klikTautan, mengikuti

ER% chart per tanggal

$$ER\% = (SUM(jumlahBersihInteraksi) \div SUM(jangkauan)) \times 100$$

(= 0 kalau jangkauan = 0)

11. Produk (Packages)

Tab analisa per produk (content_name). Menunjukkan revenue, transaksi, unique users, dan range harga per produk.

Per produk (grouped by content_name)

Metric	Formula
revenue	SUM(revenue) baris dengan content_name ini
transactions	COUNT baris dengan content_name ini
uniqueUsers	COUNT(DISTINCT identitas) — pakai email, kalau kosong pakai phone, kalau kosong pakai full_name, ka
aov	revenue ÷ transactions (= harga rata-rata per line item)
arppu	revenue ÷ uniqueUsers (= rata-rata revenue per user unik)
minPrice	Harga termurah yang pernah dijual
maxPrice	Harga tertinggi yang pernah dijual
avgPrice	Rata-rata harga = revenue ÷ transactions (= aov)
lowTrx	Jumlah transaksi di harga minPrice
highTrx	Jumlah transaksi di harga maxPrice
durationDays	CEIL((maxDate – minDate) / 86400000), minimum 1 hari
durationLabel	≥365 → "X.X Tahun" ≥30 → "X.X Bulan" else → "N Hari"

Sorting & pagination

- Sorted by revenue desc (paling laris di atas)
- Pagination 10 per halaman
- Search filter by nama produk (substring case-insensitive)

12. Kode Promo

Tab analisa kode promo. Setiap transaksi punya kolom promo_code yang di-klasifikasi ke 7 kategori.

Parsing kode promo

```
Cara baca kolom promo_code:  
null / "" / "[]" / "null" → dianggap kosong (isEmpty=true)  
"[X,Y]" → strip bracket, split koma → ["X", "Y"]  
"X" → 1 kode  
"X,Y" → 2 kode (X = mainCode, Y = resellerCode)  
  
normalisasi tiap kode: buang spasi, dash, underscore → uppercase  
"DISKON-50" → "DISKON50"  
"promo 100" → "PROMO100"
```

Klasifikasi kategori (urutan priority)

1. Kalau kode kosong → kategori "Tanpa Kode"
2. Cek dulu di promo_code_rules user (kalau dia bikin aturan custom)
Match exact: lowercase(app) + ":" + normalizedCode
3. Kalau gak match user rule → cek hardcoded regex per app:
cerebrum / jadian / jadibumn / jadipolisi / jadiprajurit / jadisekdin
→ kategori: Sales / Marketing / Artikel / Aplikasi / Live
4. Kalau gak match apa-apa → kategori "Lainnya"

Tabel 1: Stats per kategori

Kategori	Cara hitung
count	COUNT transaksi di kategori ini
revenue	SUM(revenue) di kategori ini
pct (%)	$(\text{revenue} \div \text{grand_total_revenue}) \times 100$
Reseller row	COUNT/SUM transaksi yang punya resellerCode (informasi tambahan, sudah dihitung di kategori main)

Tabel 2: Stats per kode unik

```
Display key per transaksi:  
mainCode kalau gak ada resellerCode  
mainCode + "+RES" kalau punya resellerCode  
  
Group by displayKey:  
count = COUNT transaksi  
revenue = SUM(revenue)
```

Detail card (search spesifik kode)

User search kode tertentu → sistem cari rows yang display-key-nya mengandung query (setelah uppercase + buang non-alphanumeric). Lalu di-breakdown per source_app dengan $\text{pct} = \text{revenue} \div \text{totalRev} \times 100$.

13. Optimasi Harga

Sub-tab yang ngasih rekomendasi pricing berdasarkan data historis.

Filter (Price Suggestion)

- Platform (mis. JADIASN)
- Content name substring (mis. "3 bulan")

Statistik dasar

```
minPrice = harga terendah
maxPrice = harga tertinggi
avgPrice = rata-rata
mostBoughtPrice = harga paling sering dibeli (modus)

lowCount = COUNT transaksi dengan price ≤ minPrice × 1.1
avgCount = COUNT transaksi dengan avgPrice × 0.9 ≤ price ≤ avgPrice × 1.1
highCount = COUNT transaksi dengan price ≥ maxPrice × 0.9
```

Rekomendasi pricing

```
valuePrice = round((avgPrice + mostBoughtPrice) ÷ 2, kelipatan 1000)
campaignPrice = round((minPrice + valuePrice) ÷ 2.2, kelipatan 1000)
anchorPrice = round(maxPrice × 1.15, kelipatan 1000)

Sanity adjust:
kalau campaignPrice < valuePrice × 0.6 → campaignPrice = round(valuePrice × 0.7)
kalau campaignPrice ≥ valuePrice → campaignPrice = valuePrice - 5000
kalau valuePrice ≥ anchorPrice → anchorPrice = valuePrice + 10000
```

Konsep:

- **Campaign price:** harga promo/diskon (paling murah)
- **Value price:** harga normal (rekomendasi default)
- **Anchor price:** harga premium (untuk paket high-end)

Distribusi pembeli

```
5 buckets dengan lebar sama:
step = (maxPrice - minPrice) ÷ 5
bucket[i] = [minPrice + i × step, minPrice + (i+1) × step)
bucket terakhir inclusive di kanan

count per bucket = jumlah transaksi di range itu
```

Pricing Comparison (tab kedua)

Grouped by tahun ATAU year_month (toggle), optionally × source_app.

```
Per grup:
revenue = SUM(revenue)
transactions = COUNT
uniqueUsers = COUNT DISTINCT identitas
aov = revenue ÷ transactions
arppu = revenue ÷ uniqueUsers

Prices di-sort ascending:
```

```
minPrice = prices[0]
maxPrice = prices[N-1]
benchmarkPrice = prices[floor(N × 0.5)] (median / P50)
lowRec = prices[floor(N × 0.25)] (P25)
highRec = prices[floor(N × 0.75)] (P75)
```

14. Kalender Marsel

Tab visualisasi kalender bulan, menunjukkan paket-paket yang aktif terjual per hari untuk app yang dipilih.

Filter

```
data.filter(d.source_app.toUpperCase() === selectedApp.toUpperCase())
```

Per hari (yyyy-MM-dd)

```
packages: grouped by content_name, per produk:  
revenue = SUM(revenue) di hari itu  
transactions = COUNT transaksi  
minPrice = MIN(price)  
maxPrice = MAX(price)  
avgPrice = revenue ÷ transactions  
  
appBreakdown[source_app] = { revenue, transactions }
```

Stats bulan

```
monthTotals:  
revenue = SUM dari daily revenues  
transactions = SUM dari daily transactions  
  
uniqueBuyers (bulan terpilih):  
COUNT DISTINCT (email ?? phone ?? full_name ?? trx_id)  
  
maxDayRevenue = MAX(dayRevenue)  
topDay = hari dengan revenue tertinggi
```

AI insight (payload yang dikirim ke AI)

Untuk hari yang dipilih, sistem kompilasi: revenue, transactions, downloaders (SUM count di tanggal itu), $\text{conversion} = \text{transactions} \div \text{downloaders} \times 100$, strategi (dari Strategi & Target), konten sosmed yang aktif, skrip yang dijadwalkan, dan tugas yang due hari itu.

15. Glosarium

Istilah-istilah yang sering muncul di dashboard.

Istilah	Arti
Revenue / Sales	Total pendapatan dalam Rupiah. Berasal dari SUM(revenue) di tabel transactions.
Transaksi	Jumlah baris transaksi (= line item). 1 order dengan 3 item dihitung 3 transaksi.
Order / trx_id	Kode pesanan unik. Untuk distinct count "jumlah order" pakai trx_id.
User / Customer / Premium	Email unik (= 1 orang). Digunakan untuk menghitung jumlah USER bukan jumlah transaksi.
Downloader	Jumlah download app per hari. SUM dari kolom count di tabel downloaders.
Konversi (%)	$(\text{Jumlah transaksi} \div \text{Jumlah downloader}) \times 100$. Berapa persen yang download terus akhirnya beli.
AOV (Average Order Value)	$\text{Revenue} \div \text{Jumlah transaksi}$. Rata-rata nilai 1 line item.
ARPPU	Average Revenue Per Paying User. $\text{Revenue} \div \text{unique users}$. Rata-rata duit dari 1 customer.
Target (bulanan)	Angka target yang diset user di Strategi & Target untuk app+bulan tertentu.
Real / Actual	Angka realisasi (dari DB atau manual override).
Achievement	$(\text{Real} \div \text{Target}) \times 100$. Persen pencapaian target.
Selisih sales (signed)	$\text{Real} - \text{Target}$. Positif = surplus (Kelebihan), negatif = defisit (Kekurangan).
Flexible / Chained Target	Target harian yang menyesuaikan defisit hari sebelumnya. Range 20%-200% dari base.
Status Menunggu / Kurang / Lebih / Melampaui	Model status pencapaian harian. Lihat Bab 5 untuk detail.
Repeat Order	Email dengan ≥ 2 trx_id berbeda dalam scope filter.
Realtime	Auto-update via Supabase WebSocket. Saat ada INSERT/UPDATE/DELETE di DB, dashboard otomatis update.
IndexedDB	Database lokal di browser. Saat ini SiMarsel TIDAK pakai cache lokal — load ulang setiap session (~30s).
Promo Category	Klasifikasi kode promo: Sales / Marketing / Aplikasi / Live / Artikel / Lainnya / Tanpa Kode.
Kategori "Tanpa Kode"	Transaksi yang gak pakai promo apapun. Bukan termasuk Lainnya.
Year_month	Format "YYYY-MM" (mis. "2026-05"). Dipakai untuk grouping dan target lookup.

--- Akhir Dokumen ---

Dokumen ini disusun berdasarkan audit kode SiMarsel pada Mei 2026. Logic perhitungan bisa berubah seiring update aplikasi. Untuk versi terbaru, cek branch main di repository GitHub.